

[Chap 14] 자연어 처리 기초



개요

- 뉴스, 고객 리뷰 등 언어로 되어 있는 데이터에 비즈니스에 중요한 정보가 들어 있는 경우가 많음
 - 언어 데이터는 대량의 텍스트로 표현된 비정형 데이터(unstructured data)
 - 빅데이터의 대표적인 유형
- 자연어(natural language) 데이터
 - 일상의 언어 표현으로 되어 있는 데이터
- NLP(Natural Language Processing)
 - 자연어의 분석과 처리를 위한 기술을 통틀어 가리킴

BOW 모델

- 컴퓨터로 자연어를 처리하기 위한 첫 번째 단계는 언어를 숫자로 바꾸는 것임
- 다음 문장을 숫자로 바꾸면?

이 영화 완전 강추 진짜 다른 영화

- 위 문장의 단어별 빈도

〈표 14-1〉 리뷰 문장의 빈도 벡터

이	영화	완전	강추	진짜	다른
1	2	1	1	1	1

3

BOW 모델 (계속)

- BOW(Bag Of Words) 모델
 - 문장 또는 문서에서 나타난 단어들을 순서를 무시하고 횟수만 세어서 숫자로 바꾸는 모델
 - 앞의 문장의 BOW 모델은 (1, 2, 1, 1, 1, 1)이 됨



[그림 14-1] BOW 모델의 개념

4

BOW 모델 구축 실습

- 긴 문서의 예: 제6차 국가정보화기본계획
 - NIA 홈페이지(www.nia.or.kr)에서 다운로드 가능
 - 일반 텍스트로 저장
 - 텍스트 인코딩을 'UTF-8'로 설정
- 파일 읽어오기

```
1 f = open('정보화기본계획.txt', 'r', encoding='utf8')
2 texts = []
3 while True:
4     line = f.readline()
5     if not line: break
6     texts.append(line)
7 text = ''.join(texts)
8 print(text)
```

지능정보사회 구현을 위한
제6차 국가정보화 기본계획(2018~2022)

01

H

추진배경 :
초연결 지능화

패러다임 전환

CHAPTER

01

추진배경

초연결 지능화 패러다임 전환

CHAPTER

모든 것이 서로 연결되고, 보다 지능화된
• 인터넷과 컴퓨터 기반의 '정보화'사회를 넘
'능화' 사회로 빠르게 진입 ...

5

텍스트 전처리

- 텍스트를 단어로 분리
 - 일반적으로 형태소 분석기를 활용하여 명사 또는 형용사 등의 원형을 추출
 - 한국어 형태소분석기: konlpy 라이브러리 또는 Khaii(카카오) 등 활용
 - 여기서는 단순히 띄어쓰기 단위로 분리하여 실습
- 단어 구분을 위한 텍스트 전처리(text preprocessing)
 - 한국어 텍스트의 단어 구분: 공백, 줄바꿈 문자(\n), 마침표, 물음표, 느낌표, 쉼표, 줄표(-), 두점(:), 물결표(~), 슬래시(/) 등
 - 이외에 괄호, 따옴표, 화살표, 참고표(※) 등의 다양한 문장 부호가 있으며, 공문서에 서는 가운뎃점(.) 및 다양한 불릿 문자(■, ●, ► 등)도 사용
 - 단어를 구분하는 문자를 모두 공백 문자로 교체

```
1 special = ['\n', ',', '.', ':', '~', '/', '!', '?', '※',
2           '(', ')', '[', ']', '"', "'", '<']
3 for c in special:
4     text = text.replace(c, ' ')
5 print(text)
```

지능정보사회 구현을 위한 제6차 국가정보화 기본계획 2018 2022 01 H 추진배
경 초연결 지능화 패러다임 전환 CHAPTER 01 추진배경 초연결 지능화 패러다
임 전환 CHAPTER 모든 것이 서로 연결 되고 보다 지능화된 사회로 진입 인터넷
과 컴퓨터 기반의 정보화 사회를 넘어 전 세계는 초연결 Hyper Connectivity 지능화 사
회로 빠르게 진입 ...

6

단어별 빈도 계산

- 텍스트를 공백 기준으로 분리
 - `split(' ')` : 공백 기준으로 분할한 토큰(token)의 리스트 생성

```
1 wordList = text.split(' ')
2 print(wordList)
```

```
['지능정보사회', '구현을', '위한', ' ', ' ', '제6차', '국가정보화', '기본계획', '2018', '2022',
 ' ', ' ', ' ', ' ', ' ', '01', ' ', ' ', ' ', 'H', ' ', ' ', ' ', '추진배경', ' ', ' ', ' ', '초연결', '지능
 화', ' ', ' ', '패러다임', '전환', ' ', ' ', ' ', 'CHAPTER', ' ', ' ', ' ', '01', ' ', ' ', ' ', '추진배경',
 ' ', '초연결', '지능화', '패러다임', '전환', ' ', ' ', ' ', 'CHAPTER', ' ', ' ', ' ', ' ', '모든', '것
 이', '서로', '연결되고', ' ', '보다', '지능화된', '사회로', '진입', ' ', ' ', ' ', ' ', '인터넷과', '컴
 퓨터', '기반의', ' ', '정보화', '사회를', '넘어', ' ', '전', '세계는', ' ', '초연결', 'Hyper', ...
```

- 토큰별 등장 빈도 계산
 - 딕셔너리(dictionary) 자료형 활용
 - (키, 값) 형태로 저장됨

```
1 words = {}
2 for w in wordList:
3     if len(w)==0: continue
4     if w not in words: words[w] = 1
5     else: words[w] += 1
6 print(words)
```

(1행) 빈 딕셔너리 생성
(2행) 문서의 각 단어를 w에 저장
(3행) w가 길이가 0인 단어이면 넘김
(4행) w가 딕셔너리 words에 처음 나온 단어이면 해당 단어를 새로운 키로 등록하고 값을 1로 설정
(5행) w가 이전에 나온 적이 있는 단어이면 해당 단어의 값을 하나 증가

```
{'지능정보사회': 30, '구현을': 9, '위한': 247, '제6차': 8, '국가정보화': 30, '기본계획': 16,
 '2018': 9, '2022': 8, '01': 3, 'H': 19, '추진배경': 2, '초연결': 20, '지능화': 90, '패러다임':
 5, '전환': 24, 'CHAPTER': 13, '모든': 12, '것이': 1, ...}
```

7

정렬 및 불용어 처리

- 딕셔너리의 단어들을 빈도값이 높은 것부터 내림차순으로 정렬

```
1 sortedWords = sorted(words.items(), key=(lambda x: x[1]), reverse=True)
2 print(sortedWords)
```

- 등, '및', '위한' 같이 주제와 관계없이 문장에 늘 등장하는 일반적인 단어들, 즉 불용어(stopword)들이 상위 빈도를 차지하고 있어 제거 필요

```
[('등', 444), ('및', 431), ('위한', 247), ('구축', 197), ('데이터', 163), ('지원', 148), ('기
 반', 148), ('추진', 136), ('개발', 135), ('서비스', 128), ('기술', 107), ('확대', 106), ('통
 해', 105), ('과기정통부', 105), ('강화', 104), ('디지털', 94), ('등을', 92), ('지능화', 90),
 ('지능형', 86), ('빅데이터', 85), ('통한', 84), ('활용', 83), ('1', 82), ('AI', 74), ('위해',
 74), ('수', 73), ('2', 72), ('조성', 72), ('마련', 71), ('클라우드', 71), ('17', 68), ('3', 67),
 ('확산', 66), ('4차', 64), ('산업', 63), ('IoT', 63), ('교육', 62), ('맞춤형', 58), ('18',
 56), ('스마트', 56), ('분야', 54), ('플랫폼', 51), ('인공지능', 50), ('국가', 48), ('네트워
 크', 47), ('4', 47), ('분석', 46), ('있는', 46), ('필요', 45), ('글로벌', 44), ('인프라', 44),
 ('ICT', 43), ('대한', 43), ('16', 42), ('제고', 42), ('활성화', 42), ...]
```

- 불용어 제거 부분 추가

```
15 stopwords=['등', '및', '위한', '통해', '등을', '통한', '활용', '위해',
16 '수', '있는', '필요', '대한', '관련', '있도록', '다양한',
17 '1', '2', '3', '4', '5', '6', '7', '8', '9', '0',
18 '13', '14', '15', '16', '17', '18', '19', '20', '21', '22']
19 words = {}
20 for w in wordList:
21     if len(w)==0: continue
22     if w in stopwords: continue
23     if w not in words: words[w] = 1
24     else: words[w] += 1
25
26 sortedWords = sorted(words.items(), key=(lambda x: x[1]), reverse=True)
27 print(sortedWords)
```

```
[('구축', 197), ('데이터', 163), ('지원', 148), ('기반', 148), ('추진', 136), ('개발', 135),
 ('서비스', 128), ('기술', 107), ('확대', 106), ('과기정통부', 105), ('강화', 104), ('디지털',
 94), ('지능화', 90), ('지능형', 86), ('빅데이터', 85), ('AI', 74), ('조성', 72), ('마련', 71),
 ('클라우드', 71), ('확산', 66), ('4차', 64), ('산업', 63), ('IoT', 63), ('교육', 62), ('맞춤형',
 58), ('스마트', 56), ('분야', 54), ('플랫폼', 51), ('인공지능', 50), ...]
```

8

BOW의 시각화 실습

- wordcloud 패키지 설치

```
C:\WINDOWS\system32>pip install wordcloud_
```

- 다음과 같은 오류 발생시 Microsoft Visual C++ Build Tools의 추가 설치 필요

```
error: Microsoft Visual C++ 14.0 is required. Get it with "Microsoft Visual C++ Build Tools": https://visualstudio.microsoft.com/downloads/
```

→ <https://visualstudio.microsoft.com/ko/downloads> 에서 커뮤니티 버전 설치

- 폰트 설치
 - 공개 폰트 'Kopub World 돋움체 M5'의 파일 'KoPubWorld Dotum Medium.ttf'를 코드가 있는 폴더에 함께 복사
 - <https://gongu.copyright.or.kr/gongu/main/main.do>

9

워드클라우드(wordcloud)

- 워드클라우드 생성 (전체 코드: ex14-2.py)

```
1 from wordcloud import WordCloud
2 import matplotlib.pyplot as plt
3
4 wordcloud = WordCloud(font_path='./KoPubWorld Dotum Medium.ttf',
5     background_color='white', width=1024, height=1024,
6     max_font_size=100, min_font_size=10, max_words=300,
7     colormap='Dark2').generate_from_frequencies(words)
8 plt.figure()
9 plt.imshow(wordcloud, interpolation='bilinear')
10 plt.axis('off')
11 plt.show()
```

옵션별 의미

〈표 14-2〉 워드클라우드의 옵션별 의미

옵션명	의미
font_path	워드클라우드에 사용할 폰트파일
background_color	워드클라우드 배경색
width	워드클라우드 가로크기 (픽셀 수)
height	워드클라우드 세로크기 (픽셀 수)
max_font_size	가장 큰 글자 크기
min_font_size	가장 작은 글자 크기
max_words	워드클라우드에 나타낼 최대 단어 갯수
colormap	워드클라우드에 사용할 색상표 이름1

워드클라우드 시각화 결과

- 시각화된 그림을 보면 직관적으로 이 문서의 중심 주제를 파악할 수 있음
 - 문서에 기술된 정책의 중심이 데이터(빅데이터), 클라우드, 인공지능 등으로 나타남



빈도 벡터와 DTM

- 다음 3개의 문장간 유사도 측정

문장 A: 이 영화 완전 강추 진짜 다른 영화

문장 B: 진짜 강추 완전 좋은 영화

문장 C: 자막 진짜 엉망

- 문서-단어 행렬(DTM: Document-Term Matrix)
 - 전체 문장 A, B, C의 BOW를 통합

〈표 14-3〉 리뷰 문장들의 DTM

단어	이	영화	완전	강추	진짜	다른	좋은	자막	엉망
문장 A	1	2	1	1	1	1	0	0	0
문장 B	0	1	1	1	1	0	1	0	0
문장 C	0	0	0	0	1	0	0	1	1

13

벡터 내적과 문장 유사도

- 벡터의 내적(inner product)

$$\mathbf{x} \cdot \mathbf{y} = x_1y_1 + x_2y_2 + x_3y_3 = |\mathbf{x}| |\mathbf{y}| \cos(\theta) \quad (\text{식 14-1})$$

- 두 벡터가 원점으로부터 비슷한 방향에 놓여있을수록 내적은 커지고, 원점을 중심으로 서로 직교한다면 0, 반대방향을 가리키고 있으면 내적은 - 값이 됨

- 문장 A와 B, C의 BOW 벡터간 내적

$$\mathbf{a} \cdot \mathbf{b} = (1 \times 0) + (2 \times 1) + (1 \times 1) + (1 \times 1) + (1 \times 1) + (1 \times 0) + (0 \times 1) + (0 \times 0) + (0 \times 0) = 5 \quad (\text{식 14-2})$$

$$\mathbf{a} \cdot \mathbf{c} = (1 \times 0) + (2 \times 0) + (1 \times 0) + (1 \times 0) + (1 \times 1) + (1 \times 0) + (0 \times 0) + (0 \times 1) + (0 \times 1) = 1$$

- 문장 A는 B와는 유사도가 높지만 C와는 유사도가 낮음이 반영

- 단순 내적의 문제

문장 C': 자막 진짜 엉망 자막 진짜 엉망 자막 진짜 엉망 자막 진짜 엉망 자막 진짜
엉망 자막 진짜 엉망

- 의미상 C와 C'는 동일하지만 $\mathbf{c}' = (0, 0, 0, 0, 6, 0, 0, 6, 6)$ 이므로 $\mathbf{a} \cdot \mathbf{c}' = 6$ 이 됨

코사인 유사도

- 코사인 유사도(cosine similarity)

$$\text{similarity}(x, y) = \cos(\theta) = \frac{x \cdot y}{|x||y|} \quad (\text{식 14-3})$$

- 벡터의 내적을 각 벡터의 절대값으로 나눈 값
- 문서의 길이에 의한 영향을 배제하고 단어 등장 패턴의 유사성 측정 가능

- 문장 A와 문장 B, C의 코사인 유사도

$$\begin{aligned} \text{similarity}(a, b) &= \frac{a \cdot b}{|a||b|} = \frac{5}{3 \times 2.24} = 0.75 \\ \text{similarity}(a, c) &= \frac{a \cdot c}{|a||c|} = \frac{6}{3 \times 10.39} = 0.19 \end{aligned} \quad (\text{식 14-5})$$

- 문서 길이에 영향 받지 않고 의미적 유사도에 부합하는 값을 제공

15

코사인 유사도의 한계

- 다음 3개의 문장 고려

문장 A: 이 영화 완전 강추 진짜 다른 영화

문장 B: 진짜 강추 완전 좋은 영화

문장 C: 이 영화 영화 자막 완전 진짜 완전 엉망

- 코사인 유사도의 문제

〈표 14-4〉 수정된 리뷰 문장들의 DTM

단어	이	영화	완전	강추	진짜	다른	좋은	자막	엉망
문장 A	1	2	1	1	1	1	0	0	0
문장 B	0	1	1	1	1	0	1	0	0
문장 C	1	2	2	0	1	0	0	1	1

$$\begin{aligned} \text{similarity}(a, b) &= \frac{a \cdot b}{|a||b|} = \frac{5}{3 \times 2.23} = 0.75 \\ \text{similarity}(a, c) &= \frac{a \cdot c}{|a||c|} = \frac{8}{3 \times 3.46} = 0.77 \end{aligned}$$

- 문장 A와 C가 의미상 반대이지만 코사인 유사도는 오히려 높게 나옴
- 문장 A와 C의 유사도를 높게 만든 '영화', '완전', '진짜' 등의 단어는 모든 리뷰에서 등장하는 흔한 단어로서 의미 있는 정보를 담고 있다고 보기 어려움

TF-IDF의 개념

- TF-IDF(Term Frequency-Inverse Document Frequency)

$$TF-IDF(w, d, D) = tf(w, d) \times idf(w, D) = tf(w, d) \times \log\left(\frac{N}{df(w, D)}\right)$$

- 단어 빈도를 해당 단어가 등장하는 문서의 빈도(의 로그값)로 나누는 개념
- 여러 문서에 공통적으로 등장하는 단어는 TF-IDF의 값이 낮아짐

- 문서 A에서 '완전'의 TF-IDF

- TF = 1, DF = 3이므로,

$$TF-IDF('완전', A, D) = tf('완전', D) = 1 \times \log\left(\frac{3}{3}\right) = 0$$

- 즉, '완전' 이라는 단어는 정보량이 없음을 반영

17

TF-IDF의 계산

- 문서 d 내에서 단어 w의 빈도를 계산하는 함수

```
1 def tf(w, d):
2     cnt = 0
3     for t in d:
4         if t==w: cnt+=1
5     return cnt
```

- 특정 단어 w가 등장한 문서 개수를 계산하는 함수 (D: 문서들의 리스트)

```
1 def df(w, D):
2     cnt = 0
3     for sent in D:
4         if w in sent: cnt+=1
5     if cnt==0: cnt = 1
6     return cnt
```

18

TF-IDF의 계산 (계속)

- TF-IDF를 계산하는 함수

```
1 def tfidf(w,d,D):
2     N=len(D)
3     return tf(w,d)*np.log(N/df(w,D))
```

- 문장 A, B, C의 각 단어에 대한 TF-IDF 값 계산

```
20 sent_A = '이 영화 완전 강추 진짜 다른 영화'.split(' ')
21 sent_B = '진짜 강추 완전 좋은 영화'.split(' ')
22 sent_C = '이 영화 영화 자막 완전 진짜 완전 엉망'.split(' ')
23 corpus=[sent_A, sent_B, sent_C]
24 print(corpus)
25
26 print('<Sentence A>')
27 for w in sent_A:
28     print(w, tfidf(w,sent_A,corpus))
29 print('<Sentence B>')
30 for w in sent_B:
31     print(w, tfidf(w,sent_B,corpus))
32 print('<Sentence C>')
33 for w in sent_C:
34     print(w, tfidf(w,sent_C,corpus))
```

19

TF-IDF의 계산 (계속)

- 전체 문서에 대한 TF-IDF 행렬

〈표 14-5〉 단어별 TF-IDF 값

문서	이	영화	완전	강추	진짜	다른	좋은	자막	엉망
A	0.41	0	0	0.41	0	1.10	0	0	0
B	0	0	0	0.41	0	0	1.10	0	0
C	0.41	0	0	0	0	0	0	1.10	1.10

- TF-IDF 벡터의 코사인 유사도

$$\begin{aligned} \mathbf{a}' &= (0.41, 0, 0, 0.41, 0, 1.10, 0, 0, 0) & \text{similarity}(\mathbf{a}', \mathbf{b}') &= \frac{\mathbf{a}' \cdot \mathbf{b}'}{\|\mathbf{a}'\| \|\mathbf{b}'\|} = \frac{0.1644}{1.24 \times 1.17} = 0.11 \\ \mathbf{b}' &= (0, 0, 0, 0.41, 0, 0, 1.10, 0, 0) \\ \mathbf{c}' &= (0.41, 0, 0, 0, 0, 0, 0, 1.10, 1.10) & \text{similarity}(\mathbf{a}', \mathbf{c}') &= \frac{\mathbf{a}' \cdot \mathbf{c}'}{\|\mathbf{a}'\| \|\mathbf{c}'\|} = \frac{0.1644}{1.24 \times 1.61} = 0.08 \end{aligned}$$

- 실제 의미와 마찬가지로 A와 B의 유사도가 더 높게 나타남

TF-IDF의 계산 (계속)

- TF-IDF 데이터 준비

```
1 wordSet = ['이','영화','완전','강추','진짜','다른','좋은','자막','영망']
2 tfidf_a=[]
3 tfidf_b=[]
4 tfidf_c=[]
5 for w in wordSet: tfidf_a.append(tfidf(w,sent_A,corpus))
6 for w in wordSet: tfidf_b.append(tfidf(w,sent_B,corpus))
7 for w in wordSet: tfidf_c.append(tfidf(w,sent_C,corpus))
8
9 print('tfidf_a = ', tfidf_a)
10 print('tfidf_b = ', tfidf_b)
11 print('tfidf_c = ', tfidf_c)
```

```
[0.4054651081081644, 0.0, 0.0, 0.4054651081081644, 0.0, 1.0986122886681098, 0.0, 0.0, 0.0]
[0.0, 0.0, 0.0, 0.4054651081081644, 0.0, 0.0, 1.0986122886681098, 0.0, 0.0]
[0.4054651081081644, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0986122886681098, 1.0986122886681098]
```

21

TF-IDF의 계산 (계속)

- 벡터의 절대값 계산

```
1 def magnitude(v):
2     m=0
3     for x in v: m += x**2
4     return (np.sqrt(m))
```

- 벡터간 코사인 유사도 계산

```
1 def similarity(a,b):
2     dot = np.dot(a,b)
3     ma = magnitude(a)
4     mb = magnitude(b)
5     return(dot/(ma*mb))
```

- 문서 A, B, C의 TF-IDF 벡터간 유사도 계산

```
1 print('Magnitude of tfidf_a = ', magnitude(tfidf_a))
2 print('Magnitude of tfidf_b = ', magnitude(tfidf_b))
3 print('Magnitude of tfidf_c = ', magnitude(tfidf_c))
4 print('similarity(tfidf_a, tfidf_b) = ', similarity(tfidf_a, tfidf_b))
5 print('similarity(tfidf_a, tfidf_c) = ', similarity(tfidf_a, tfidf_c))
```

전체 코드: ex14-4.py

22

감정분석 (SENTIMENT ANALYSIS)

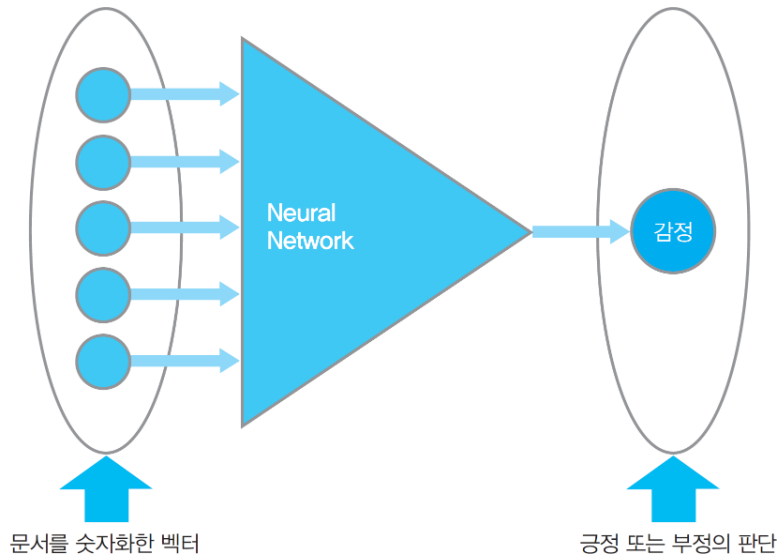
23

감정분석(Sentiment Analysis)

- 자연어 텍스트에 나타난 감정을 이해하는 것
- 다양한 활용
 - 주식 게시판 등을 통한 주식 종목에 대한 투자자 정서 파악
 - 미디어에 나타난 특정 기업에 대한 시장 정서
 - 고객들의 제품이나 서비스에 대한 반응
 - 소프트웨어나 기계 장비의 로그 파일을 통한 상태 파악 등
- 감정분석 기법의 종류
 - 통계적 방법
 - 긍부정어 사전 기반
 - SO-PMI (Semantic-Oriented Pointwise Mutual Information)
 - 인공신경망 기반

인공신경망을 이용한 감성분석

- 신경망 구조
 - 입력: BOW 등을 활용하여 문서를 숫자화한 벡터 (BOW 등)
 - 출력: 긍정/부정을 1과 0으로 표현



[그림 14-3] 감성분석을 위한 인공신경망 구조

25

IMDB movie review data

- 영화 리뷰 데이터로서, 단어를 숫자로 바꾸어놓은 리뷰 내용과 해당 리뷰가 긍정적인지 부정적인지를 나타낸 정답값이 함께 들어 있음
 - 단어는 숫자로 나타나 있으며, 단어 번호는 등장 빈도 순서로 매겨져 있음

예제 ex14-5.py

```
1 from tensorflow import keras
2 from tensorflow.keras.datasets import imdb
3 import numpy as np
4
5 (train_data, train_labels), (test_data, test_labels) = imdb.load_data()
6 print(type(train_data))
7
8 print(train_data.shape, train_labels.shape)
9 print(test_data.shape, test_labels.shape)
10 print('Train data 0 = ', train_data[0])
11 print('Train label 0 = ', train_labels[0])
```

Train data 0 = [1, 14, 22, 16, 43, 530, 973, 1622, 1385, 65, 458, 4468, 66, 3941, 4, 173, 36, 256, 5, 25, 100, 43, 838, 112, 50, 670, 22665, 9, 35, 480, 284, 5, 150, 4, 172, 112, 167, 21631, 336, 385, 39, 4, 172, 4536, 1111, 17, 546, 38, 13, 447, 4, 192, 50, 16, 6, 147, 2025, 19, 14, 22, 4, 1920, 4613, 469, 4, 22, 71, 87, 12, 16, 43, 530, 38, 76, 15, 13, 1247, 4, 22, 17, 515, 17, 12, 16, 626, 18, 19193, 5, 62, 386, 12, 8, 316, 8, 106, 5, 4, 2223, 5244, 16, 480, 66, 3785, 33, 4, 130, 12, 16, 38, 619, 5, 25, 124, 51, 36, 135, 48, 25, 1415, 33, 6, 22, 12, 215, 28, 77, 52, 5, 14, 407, 16, 82, 10311, 8, 4, 107, 117, 5952, 15, 256, 4, 31050, 7, 3766, 5, 723, 36, 71, 43, 530, 476, 26, 400, 317, 46, 7, 4, 12118, 1029, 13, 104, 88, 4, 381, 15, 297, 98, 32, 2071, 56, 26, 141, 6, 194, 7486, 18, 4, 226, 22, 21, 134, 476, 26, 480, 5, 144, 30, 5535, 18, 51, 36, 28, 224, 92, 25, 104, 4, 226, 65, 16, 38, 1334, 88, 12, 16, 283, 5, 16, 4472, 113, 103, 32, 15, 16, 5345, 19, 178, 32]

Train label 0 = 1

각 숫자는 해당 영어 단어에 대응

1: 긍정
0: 부정

26

숫자를 단어로 해석

- 단어 인덱스

```
13 word_index = imdb.get_word_index()
14 print(word_index['the'], word_index['and'], word_index['a'])
```

1 2 3

- IMDB dataset에 나타난 단어 번호 = word_index + 3
 - 1~3은 특수한 의미로 할당 (1 : Begin of Sentence, 2 : Unknown word 등)
 - 예) the = 4, and = 5, a = 6
- 예) I love you = [10, 116, 22]

- 숫자를 단어로 해석하는 코드

```
numToWord = {}
numToWord[0]=''
numToWord[1]='<시작>'
numToWord[2]='<미등록>'
numToWord[3]=''
for (word, num) in word_index.items():
    numToWord[num+3] = word
print(numToWord[4], numToWord[5], numToWord[6])

for i in train_data[0]: print(numToWord[i], end=' ')
```

the and a

<시작> this film was just brilliant casting location scenery story direction everyone's really suited the part they played and you could just imagine being there robert redford's is an amazing actor and now the same being director norman's father came from the same scottish island as myself so i loved the fact there was a real connection with this film the witty remarks throughout the film were great it was just brilliant so much that i bought the film as soon as it was released for retail and would recommend it to everyone to watch and the fly fishing was amazing really cried at the end it was so sad and you know what they say if you cry at a film it must have been good and this definitely was also congratulations to the two little boy's that played the part's of norman and paul they were just brilliant children are often left out of the praising list i think because the stars that play them all grown up are such a big profile for the whole film but these children are amazing and should be praised for what they have done don't you think the whole story was so lovely because it was true and was someone's life after all that was shared with us all

27

리뷰 문장의 벡터화

- 리뷰 문장을 BOW 모델로 벡터화

- 단어 개수를 1000으로 고정, 고빈도 단어 1000개만 고려하고 이후 단어는 무시

```
1 def vectorize(review_data):
2     vec=np.zeros(1000, dtype='float32')
3     for x in review_data:
4         if x>=1000: continue
5         vec[x]+=1
6     return vec
```

- 특정 리뷰를 BOW로 변환하는 예

```
print(vectorize(train_data[0]))
```

```
[ 0.  1.  0.  0. 15.  9.  3.  2.  3.  1.  0.  0.  6.  3.  3.  4. 11.  3.
  3.  2.  0.  1.  6.  0.  0.  4.  3.  0.  2.  0.  1.  0.  3.  2.  0.  1.
  ...  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.]
```

- 단어 빈도는 큰 값이 될 수 있음을 확인

28

학습 데이터 구성

- BOW 벡터의 요소값을 0과 1 사이로 정규화 필요
 - 최대값으로 나누면 1, 2회 등 낮은 빈도값은 너무 작은 값이 되는 문제
 - 최대값으로 나누는 대신, 0→0, 1→0.25, 2→0.5, 3→0.75, 4 이상→1로 변환

```
1 def vectorize(review_data):
2     vec=np.zeros(1000, dtype='float32')
3     for x in review_data:
4         if x>=1000: continue
5         if vec[x]<1: vec[x]+=0.25
6     return vec
```

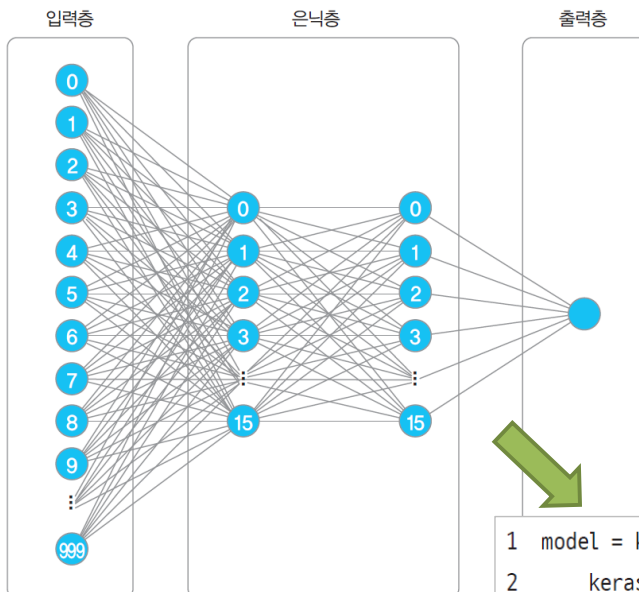
- 모든 학습 데이터를 BOW로 변환

```
1 train_input = np.zeros((len(train_data), 1000))
2 for i, review in enumerate(train_data):
3     train_input[i] = vectorize(review)
```

29

신경망 설계

- 입력노드 1000개, 출력뉴런 1개, 은닉층은 각 15개 뉴런의 2계층으로 구성



[그림 14-4] 감성분석 인공신경망 설계

```
1 model = keras.Sequential([
2     keras.layers.Dense(16, activation='relu', input_shape=(1000,)),
3     keras.layers.Dense(16, activation='relu'),
4     keras.layers.Dense(1, activation='sigmoid')])
```

학습 및 예측

- 학습을 위한 최적화 알고리즘과 손실함수 설정

```
model.compile(optimizer='adam', loss='binary_crossentropy')
```

- 학습용 데이터와 정답을 통해 학습 수행

```
model.fit(train_input, train_labels, epochs=3)
```

- 테스트 데이터를 신경망 입력에 맞는 형태로 변환

```
1 test_input = np.zeros((len(test_data), 1000))
2 for i, review in enumerate(test_data):
3     test_input[i]=vectorize(review)
```

- 테스트 데이터에 대해 긍정/부정 예측

```
1 pred = model.predict(test_input)
2 print(pred)
```

- 0과 1 사이의 확률값으로 도출되므로 0.5를 기준으로 긍정/부정의 판단 필요

31

긍정/부정 예측 및 성능평가

- 출력값 0.5를 기준으로 그 이상이면 긍정, 아니면 부정으로 판단

```
1 pred_labels=[]
2 for v in pred:
3     if v>=0.5: pred_labels.append(1)
4     else: pred_labels.append(0)
```

- 성능척도 출력

- 긍정/부정의 판별 정확도가 약 87%로 나타나, 단순한 모델에서도 상당한 성능 확인

```
1 from sklearn.metrics import classification_report
2 print(classification_report(test_labels, pred_labels))
```

	precision	recall	f1-score	support
0	0.88	0.85	0.86	12500
1	0.86	0.88	0.87	12500
accuracy			0.87	25000
macro avg	0.87	0.87	0.87	25000
weighted avg	0.87	0.87	0.87	25000

전체 코드: ex14-5.py

32

자연어 처리기술의 발전

BOW의 한계

- 단어의 빈도가 같다고 같은 의미일 수 없음
 - “딱 좋은 하루” vs. “딱 하루 좋은”
 - 순서가 의미를 가짐
- 언어 모델(language model)
 - 단어 순서에 확률을 대응시킴
 - 예) “봄꽃 만발한 산길을 걸어가”라는 문장이 나타날 수 있는 확률은 “산길을 봄꽃 걸어가 만발한”보다 훨씬 높을 것임
 - 예) 검색어 추천



단어 벡터화 방법의 발전

- 원-핫 인코딩(one-hot encoding)
 - 가장 단순한 방법으로서, 단어에 번호를 매기고 단어 가짓수만큼의 길이로 벡터를 정의하여, 각 단어를 해당 번호의 자리만 1이고 나머지는 0인 벡터로 변환
 - 0이 너무 많고 단어간의 연관성이 전혀 표현되지 않음
- 워드 임베딩(word embedding)
 - 단어를 의미적 유사도를 반영하여 n차원 좌표공간에 배치
 - Word2Vec, FastText, GloVe, ELMo 등
- Word2Vec (Word-to-Vector)
 - 구글에서 발표한 워드 임베딩 방법의 일종
 - 신경망을 통해 특정 단어에서 인접 단어들을 예측하거나(skip-gram), 인접 단어들에서 특정 단어를 예측하도록 하는(CBOW: Continuous Bag-Of-Words) 학습
 - 해당 신경망의 은닉층 출력값을 그 단어의 좌표로 사용
- 문서나 문장 등의 벡터화 방법도 발전
 - Doc2Vec(Document-to-Vector), Sent2Vec(Sentence-to-Vector) 등
 - 챗봇 등의 다양한 분야에 활용

35

문장의 컨텍스트 이해 기술의 발전

- Seq2Seq(Sequence-to-Sequence) 모델
 - 기계번역 등에 사용
 - 영어 문장 "I am a student"의 각 단어를 LSTM이나 GRU같은 순환신경망에 순차적으로 넣으면 신경망은 내부적으로 이 문장의 컨텍스트를 은닉층에 가지고 있게 됨
 - 컨텍스트 벡터와 더불어 "나는"이라는 한국어 단어 벡터를 입력값으로 주고 그 다음 단어를 예측하도록 하면 "학생입니다"가 순차적으로 산출되는 방식
 - 인코더-디코더 아키텍처(encoderdecoder architecture)라고 함
 - 인코더(encoder) : 컨텍스트 벡터를 만드는 부분
 - 디코더(decoder) : 이로부터 새로운 문장을 생성하는 부분
- 트랜스포머 아키텍처 (transformer architecture)
 - Seq2Seq 모델의 한계
 - 컨텍스트 벡터의 길이가 한정되어 있어 문장이 길어지면 번역 성능이 저하
 - 어텐션 (attention) 벡터의 도입
 - Seq2Seq 모델의 한계점 개선을 위해 디코더에서 다음 단어를 생성해낼 때 해당 시점에 원래 문장의 어떤 단어들에 주로 유의해야 하는지를 나타내는 나타내는 벡터
 - 트랜스포머 아키텍처의 등장
 - 어텐션 벡터만 두고 LSTM을 건너냄
 - 학습속도와 품질 향상으로 대량 텍스트에 대한 학습 가능

36

대규모 언어모델의 발전

- 트랜스포머 아키텍처에 기반하여 미리 대량의 텍스트에 대해 학습시켜 둔 언어모델
 - GPT(Generative Pre-trained Transformer)
 - OpenAI 재단에서 발표
 - GPT(2018), GPT-2(2019), GPT-3(2020)로 확대
 - BERT(Bidirectional Encoder Representations from Transformers)
 - Google에서 2018에 발표
 - 한국어 버전
 - SKT, Kakao, Naver 등에서 발표